

Implementation Self-Assessment Test Report

Key Exchange-DKEX-x86-v1.0 (ICCS 2026-06 electronic submission)

Per the ICCS x86-architecture Implementation Self-Assessment Guideline (2026-06, CN & EN editions)

This report records, for DKEX (Key Exchange (KEX, 3-pass + SIG)) on the x86 architecture, the basic information, test environment, functional correctness, performance, resource consumption and transfer/storage sizes, plus a raw-evidence index. Verifiable technical evidence only; no security claim. Object: the will-submit electronic package. Test date: 2026-06-24.

1 Basic Information

- **Algorithm:** DKEX (PKCKEX-359777)
- **Category:** Key Exchange (KEX, 3-pass + SIG)
- **Implementation version:** v1.0 (ICCS 2026-06 electronic submission)
- **Architecture:** x86
- **Implementations:** Reference (portable ISO C99) and Optimized (x86-64 AVX2)
- **Instances & security level:** DKEX-128/-256/-512 — 128/256/512-bit classical security level

Table 1 Parameter sets

Instance	N	Q	K	Security
DKEX-128	256	3329	2	128 -bit
DKEX-256	256	3329	4	256 -bit
DKEX-512	512	7681	4	512 -bit

The signature primitive is instantiated with ML-DSA (FIPS 204, unmodified NIST Dilithium reference) via the `adkex_sig.h` pluggable interface — a standardized third-party building block, not an original contribution.

2 Test Environment

- **Hardware:** Intel Core i9-10885H @2.40 GHz (single-core measurement), 64 GiB RAM
- **OS:** Ubuntu 24.04 (WSL2, kernel 6.x)
- **Compiler / build:** GNU gcc 13.3.0; bash build.sh
- **Guideline baseline:** native Linux, gcc $\geq 8.5.0$, kernel ≥ 4.19 — met
- **Memory footprint:** process peak resident memory $\approx 1\,956$ KB (see §6); static image size in §6

- **Measurement window (UTC):** 2026-06-23T20:56:28Z → 2026-06-23T20:59:36Z

3 Versions and Compile Configuration (Registration)

3.1 Reference (ISO C99, no SIMD)

```
gcc -std=c99 -Wpedantic -Wall -Wextra -O2
    -fcommon -DDKE_FORCE_SCALAR -DDKE_MODE=<128|256|512> -DDKE_HASH=0 -DDKE_RANDOM=0
```

3.2 Optimized (x86-64 AVX2)

```
gcc -O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer
    -std=c99 -Wpedantic -Wall -Wextra -fcommon -DDKE_HASH=0 -DDKE_RANDOM=0
```

- **Target ISA:** x86-64 + AVX2
- **RNG:** deterministic DRNG from API_PKC (DKE_RANDOM=0); KAT seeds from DRNG; reproducible.
- **Hash / XOF:** SM3 only via the official auxfunc (sm3hash/pseudoXOF/pseudohash), DKE_HASH=0; no alternative hash built.
- **Third-party:** ML-DSA (FIPS 204) NIST Dilithium reference (signatures, unmodified); additionally the x86-64 AVX2 optimization reuses low-level NTT/polynomial/sampling blocks from pq-crystals / PQClean (public domain, Apache-2.0, MIT) - external third-party, outside the B.3 scope; the avx2-linux/ hand-written NTT assembly is by proposal-team member Liu Rui (within B.3). The portable Reference is plain ISO C99 with no DKE third-party code. See Implementations/LICENSES/ and dilithium/'s own license.
- **Protected files:** drng.c/.h, auxfunc.c/.h, KAT_*.c byte-for-byte unchanged.

4 Functional Test (KAT)

x86-64 (gcc): Reference + Optimized, 6 builds over 3 instances, all KAT outputs byte-identical to Test_Vectors — 6/6 PASS (Ref ≡ Opt).

5 Performance Test (average time + average cycles)

Averages (DKE_RANDOM=0, seeds from the deterministic DRNG). N=300 for KEM, N=128 for KEX (both ≥ 100 , per guideline §3.3). Each iteration is timed with clock_gettime(CLOCK_MONOTONIC) and CPU cycles via RDTSC; single core throughout.

Table 2 Full handshake — average time / average cycles

Instance	Ref (μs)	Opt (μs)	Ref (cyc)	Opt (cyc)
DKEX-128	1484.11 μs	1001.11 μs	3 605 164	2 461 035
DKEX-256	3245.92 μs	2179.76 μs	7 801 719	5 705 900
DKEX-512	4304.16 μs	3268.28 μs	10 939 769	7 989 899

The Optimized speedup is concentrated in the NTT / polynomial arithmetic (hand

assembly). The hash is SM3 via the official auxfunc (scalar C, a compliance requirement), so the end-to-end speedup is moderate.

6 Resource Consumption (static image + peak memory)

Static image size from GNU size (text+data+bss, bytes) — storage footprint of code + constants + static data; peak resident memory from `getrusage(RUSAGE_SELF).ru_maxrss` (KB) — runtime memory footprint. Single core, single process.

Table Static image and peak memory

Instance	Ref static	Opt static	Peak RSS
DKEX-128	64 250 B	188 607 B	1 956 KB
DKEX-256	65 298 B	188 456 B	1 956 KB
DKEX-512	66 570 B	181 868 B	1 956 KB

Note: the Optimized image is markedly larger than Reference due to inlined hand assembly and -flto; peak RSS is essentially flat across instances (≈ 1.8 MB), dominated by the process baseline and on-stack key material.

7 Transfer and Storage Sizes (bytes)

long-term PK/SK = 1312/2560 (128), 2592/4896 (256, 512) B per party; messages M1/M2/M3 (128) = 800/3188/2420 B; shared secret 32 B (128/256) / 64 B (512); 3 interaction rounds.

8 Conclusion

All three DKEX instances are functionally correct on the x86 architecture (KAT byte-identical to `Test_Vectors`, `Ref` $\equiv$ `Optimized`) and build reproducibly from the submitted automated scripts. Average time, average cycles, static image and peak memory, target ISA, compile parameters, RNG method and hash backend and third-party dependency are all registered above.

9 Raw Evidence Index

The following raw data and scripts ship with the self-assessment materials package and allow independent reproduction of every result in this report:

- **3-Assessment-Code-and-Data/Raw-Evidence/assess_x86.txt** — raw performance/memory output for this architecture (with start/end timestamps)
- **3-Assessment-Code-and-Data/run_assess.sh** — unified performance/memory/KAT harness (with embedded bench sources)

- **2-Algorithm-Source-Code/Implementations/.../build.sh** — per-instance deterministic build scripts (source of functional KAT)
- **3-Assessment-Code-and-Data/Test_Vectors/** — reference vectors (byte-for-byte KAT comparison targets)

All timestamps are UTC; measurement host, compiler and window are in §2.